SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO1 ASSESSMENT TOOL 2 – Agile Sprint Planning & User Story Workshop

Course Code:	CSA10 / CSA1023	Course Title:	Software Engineering
CO Assessed:	CO1	Assessment:	Assessment Tool 2 – Agile Sprint Planning & User Story Workshop
Weightage:	20%	Total Marks:	25
CO1 Statement:	Apply advanced methodologies and agile project management techniques to manage software projects effectively		
Student Name:	Kongara Jogesh	Reg. No.:	192525035
Date:	8/10/26	Duration:	60 minutes

Code of Conduct

I KONGARA JOGESH(192525035)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.
Signature of the Student:K.Jogesh

Annexure A – Agile Sprint Planning & User Story Workshop

Instructions to Students:

Each student is assigned an individual software project problem statement. Based on the given problem statement, perform an Agile Sprint Planning and User Story Workshop by applying Agile software engineering practices. Prepare a comprehensive sprint planning document covering the following aspects:

Based on the given problem statement, perform an Agile Sprint Planning and User Story Workshop by applying Agile software engineering practices. Prepare a comprehensive sprint planning document covering the following aspects:

1. Identify the Project Vision and Stakeholders

- Define the project's vision, objectives, and identify all key stakeholders involved.

2. Create Epics and User Stories

- Break down the requirements into Epics and formulate well-structured User Stories using the format: As a <user>, I want <feature>, so that <benefit>.

3. Define Acceptance Criteria

- Write clear and measurable acceptance criteria for each selected user story using the Given–When–Then format or equivalent.

4. Prioritize the Product Backlog

- Organize and prioritize the product backlog based on business value, customer needs, and project dependencies.

5. Plan the Sprint Backlog

- Select high-priority user stories for the first sprint and divide them into actionable development tasks.

6. Estimate Effort Using Story Points

- Assign story points to each selected user story using an appropriate estimation technique (e.g., Fibonacci sequence or Planning Poker) and justify your estimates.

7. Present the Sprint Goal and Expected Deliverables

- Define a clear sprint goal and describe the expected sprint deliverables, including the features to be completed at the end of the sprint.

Rubrics for Calculation

Criteria	Excellent (5 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Project Vision & Stakeholder Identification (30%)	Project vision is clear, comprehensive, and aligned with the problem statement. All relevant stakeholders and their roles are correctly identified.	Vision is clear with minor omissions. Most stakeholders are correctly identified.	Vision is partially defined. Some stakeholders are missing or incorrectly identified.	Vision is unclear or incomplete. Stakeholders are poorly identified or missing.
Epics & User Stories (25%)	Epics are well-defined and user stories are complete, follow Agile format, and fully capture functional requirements.	Epics and user stories are mostly complete with minor formatting or requirement issues.	User stories are partially complete with limited Agile structure.	User stories are incomplete, unclear, or do not follow Agile practices.
Acceptance Criteria (20%)	Acceptance criteria are specific, measurable, testable, and aligned with all user stories.	Acceptance criteria are mostly clear and testable with minor gaps.	Acceptance criteria are partially defined and lack clarity or completeness.	Acceptance criteria are missing or not aligned with user stories.
Product Backlog Prioritization & Sprint Planning (15%)	Product backlog is logically prioritized, sprint backlog is realistic, and task breakdown is complete.	Backlog prioritization and sprint planning are mostly appropriate with minor issues.	Prioritization or sprint planning is partially complete with limited justification.	Backlog prioritization and sprint planning are poorly organized or incomplete.
Story Point Estimation, Sprint Goal & Presentation (10%)	Story point estimation is well justified, sprint goal is clear, deliverables are realistic, and presentation is professional and well organized.	Estimation and sprint goal are mostly appropriate with minor inconsistencies. Presentation is clear.	Estimation or sprint goal lacks justification. Presentation is adequate but incomplete.	Estimation is unrealistic or missing. Sprint goal, deliverables, and presentation are unclear or incomplete.

Criteria	Mark
Project Vision & Stakeholder Identification (30%)	/5
Epics & User Stories (25%)	/5
Acceptance Criteria (20%)	/5
Product Backlog Prioritization & Sprint Planning (15%)	/5
Story Point Estimation, Sprint Goal & Presentation (10%)	/5
TOTAL	/25

19. Autonomous Incident Response and Root-Cause Analysis Platform

Industry Context

Complex software systems generate large volumes of logs, metrics, alerts, and deployment events. During incidents, engineers must correlate these signals quickly to identify root causes. Alert overload and fragmented observability can delay recovery and increase operational risk.

Problem Statement

Design and develop an Autonomous Incident Response and Root-Cause Analysis Platform that correlates monitoring data, deployment changes, service dependencies, and incident histories to identify probable causes and recommend safe remediation.

The platform should integrate application monitoring, centralized logging, incident management, infrastructure-as-code, and deployment systems. Automated remediation should operate within predefined safety boundaries and require human approval for high-impact changes.

Objectives

The platform should:

- Correlate metrics, logs, traces, and deployment events
- Detect and group related alerts
- Identify probable root causes
- Recommend appropriate remediation actions
- Automate low-risk recovery within predefined limits
- Maintain incident histories and audit trails
- Support human approval for high-impact actions

Expected Outcomes

The proposed platform will:

- Reduce alert overload
- Improve root-cause identification
- Reduce incident response and recovery time
- Improve reliability of production services
- Reduce repetitive operational effort
- Provide auditable and controlled automated incident response

AGILE SPRINT PLANNING & USER STORY WORKSHOP

Project selected from Problem Statement 19: Autonomous Incident Response and Root-Cause Analysis Platform. The following sections directly address the seven required assessment activities: project vision and stakeholders, epics and user stories, acceptance criteria, backlog prioritization, sprint planning, story-point estimation, and sprint goal/deliverables.

1. Project Vision and Stakeholders

1.1 Project Vision

To build a secure and controlled autonomous incident-response platform that unifies monitoring, logs, deployment changes, service dependencies, and historical incidents so that probable root causes can be identified quickly and safe remediation can be recommended or executed within predefined boundaries.

1.2 Project Objectives

- Reduce the number of duplicate and low-value alerts presented to engineers.
- Correlate metrics, logs, traces, deployment events and service dependencies into a common incident context.
- Rank probable root causes using rule-based and machine-learning-assisted analysis.
- Recommend remediation actions together with risk levels and supporting evidence.
- Automatically execute only approved low-risk actions inside predefined safety boundaries.
- Route medium- and high-impact actions for human approval.
- Maintain complete incident history, action history and audit evidence.
- Provide dashboards for incident status, root-cause confidence, service health and remediation activity.

1.3 Key Stakeholders

Stakeholder	Role / Interest	Main Needs	Influence
SRE / DevOps Engineers	Operate production services	Fast diagnosis, reliable remediation, clear evidence	High
Incident Manager	Coordinate major incidents	Incident status, ownership, escalation and closure	High
Operations Engineer	Monitor infrastructure	Service health, alerts and safe recovery actions	High
Application Developers	Maintain applications	Application logs, traces, deployment correlation	Medium
Security Team	Protect systems	Controlled automation, audit logs, approval boundaries	High
Platform / Cloud Team	Manage infrastructure	Infrastructure state, scaling and deployment safety	High
Product Owner	Own business value	Reduced downtime, measurable reliability improvement	High
System Administrator	Maintain access and configuration	RBAC, configuration and auditability	Medium
Business / Management	Monitor service reliability	Reduced operational risk and recovery time	Medium
End Customers	Consume production services	Reliable and available services	High indirect

2. Stakeholder Needs and Challenges

Stakeholder	Current Challenge	Platform Response	Success Indicator
SRE/DevOps	Alert overload and long diagnosis time	Correlation and probable-cause ranking	Lower MTTR
Incident Manager	Multiple teams and fragmented evidence	Central incident timeline and ownership	Faster coordination
Developers	Logs and deployments stored separately	Deployment-to-incident correlation	Faster defect isolation
Security	Risk of unsafe automation	Safety boundaries and approval gates	No unauthorized high-impact action
Cloud Team	Manual infrastructure recovery	Controlled remediation integration	Lower repetitive effort
Management	Limited visibility into reliability	Dashboard and reports	Improved reliability KPIs

2.1 Key Project Challenges

- High-volume and heterogeneous observability data.
- Duplicate or correlated alerts from the same underlying failure.
- Complex service dependencies that can hide the original fault.
- Deployment changes that may be causal but are difficult to correlate manually.
- Risk of incorrect automated remediation.
- Need for human approval when a proposed action has high production impact.
- Integration differences between monitoring, logging, deployment and incident-management tools.
- Need for complete auditability of automated and human decisions.

2.2 Project Constraints

- Automation must remain inside predefined safety boundaries.
- High-impact actions require authorized human approval.
- Audit records must be preserved for every action and approval.
- Existing monitoring, logging and deployment systems should be integrated rather than replaced.
- The first sprint should deliver a demonstrable vertical slice rather than attempting the complete platform.

3. System Overview and High-Level Workflow

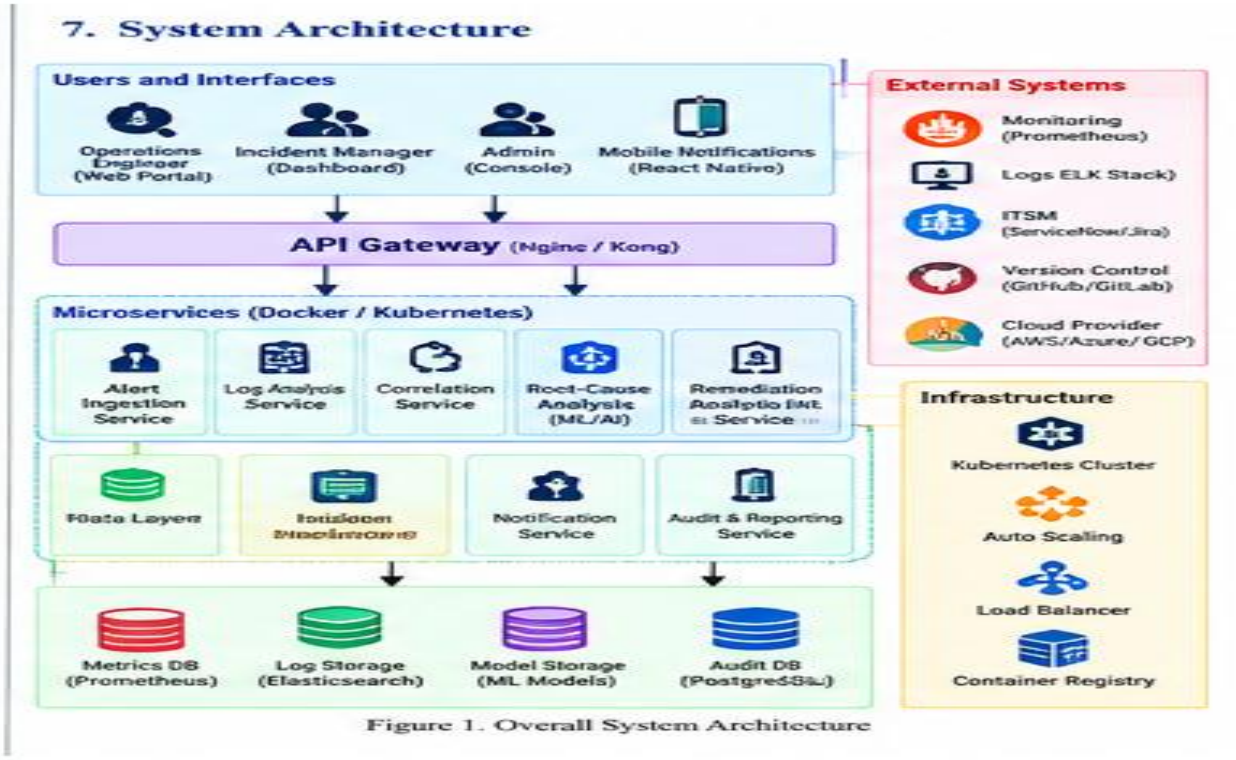


Figure 1. System Overview of the Autonomous Incident Response and Root-Cause Analysis Platform

The platform receives metrics, logs, traces and deployment events through an ingestion layer. Event correlation groups related signals, root-cause analysis evaluates probable causes, and the remediation engine recommends actions according to risk. Outputs include incident resolution information, dashboards, alerts, audit logs and post-incident reports.

3.1 End-to-End Workflow

Step	Activity	Primary Output
1	Collect monitoring and logging signals	Normalized event records
2	Group related alerts	Correlated incident candidate
3	Build incident context	Timeline + affected services
4	Analyze probable causes	Ranked root-cause candidates
5	Assess remediation risk	Low / Medium / High risk
6	Apply safety policy	Allowed / approval required / blocked
7	Execute or request approval	Action result
8	Update incident and audit records	Traceable incident history

4. Agile Epics

The problem statement is decomposed into epics so that the product backlog can be delivered incrementally. Each epic represents a coherent capability that can contain multiple user stories.

Epic ID	Epic	Description	Business Value
E1	Observability Integration	Collect metrics, logs, traces and deployment events	High
E2	Incident Detection & Correlation	Group alerts and build incident context	Very High
E3	Root-Cause Analysis	Identify and rank probable causes	Very High
E4	Remediation Recommendation	Generate safe actions with evidence and risk	Very High
E5	Controlled Automation	Execute low-risk actions and gate high-impact actions	Very High
E6	Incident Management	Track incidents, ownership, status and closure	High
E7	Audit & Governance	Record actions, approvals and policy decisions	High
E8	Dashboard & Reporting	Present operational and reliability insights	Medium
E9	Security & Access Control	RBAC, authentication and authorization	High
E10	Deployment & Operations	CI/CD, configuration, monitoring and reliability	High

5. User Stories

User stories follow the required Agile format: “As a <user>, I want <feature>, so that <benefit>.”

ID	User Story	Epic	Priority
US01	As an SRE, I want to ingest metrics, logs and traces, so that incident analysis uses a common evidence base.	E1	High
US02	As an SRE, I want related alerts grouped into one incident, so that alert overload is reduced.	E2	Very High
US03	As an incident manager, I want an incident timeline, so that I can understand the sequence of events.	E2	High
US04	As an engineer, I want probable root causes ranked, so that I can focus investigation on the most likely cause.	E3	Very High
US05	As an engineer, I want deployment changes linked to incidents, so that recent changes can be evaluated as possible causes.	E3	High
US06	As an SRE, I want remediation recommendations with evidence, so that I can choose a safe recovery action.	E4	Very High
US07	As a platform owner, I want risk levels assigned to actions, so that unsafe automation is prevented.	E5	Very High
US08	As an SRE, I want low-risk actions automatically executed, so that recovery time is reduced.	E5	Very High
US09	As an incident manager, I want high-impact actions routed for approval, so that production changes remain controlled.	E5	Very High
US10	As a security administrator, I want every action and approval recorded, so that the system remains auditable.	E7	High
US11	As a manager, I want an incident dashboard, so that I can monitor reliability and response performance.	E8	Medium
US12	As an administrator, I want role-based access control, so that only authorized users can approve or execute actions.	E9	High

6. Acceptance Criteria

Acceptance criteria are written as measurable conditions. The selected stories below form the initial sprint candidate set.

Story	Acceptance Criteria
US01	Given a configured monitoring source, When new metrics/log events arrive, Then the platform stores normalized records with source and timestamp.
US02	Given multiple alerts with a common service and time window, When correlation runs, Then related alerts are grouped into one incident candidate.
US03	Given an active incident, When an engineer opens it, Then the system displays a chronological timeline of alerts, events and deployment changes.
US04	Given sufficient incident evidence, When root-cause analysis completes, Then the system displays ranked probable causes with confidence values.
US05	Given a deployment occurred near incident onset, When correlation analyzes the incident, Then the deployment appears as related change evidence.
US06	Given a probable root cause, When remediation analysis runs, Then at least one suitable action is proposed with reason and risk level.
US07	Given a remediation action, When risk evaluation completes, Then the action is classified as low, medium or high according to configured policy.
US08	Given an action classified as low risk and inside allowed boundaries, When automation is enabled, Then the action can execute and its result is recorded.
US09	Given an action classified as high impact, When a remediation request is created, Then execution is blocked until an authorized user approves it.
US10	Given any automated or human-approved action, When it completes, Then an immutable audit entry records actor, time, action, decision and result.

7. Product Backlog Prioritization

The backlog is prioritized using business value, operational risk, dependencies and implementation readiness. Very-high-priority items create the minimum viable incident-analysis flow.

Rank	ID	Backlog Item	Priority	SP	Reason
1	US01	Observability ingestion	High	8	Foundation
2	US02	Alert correlation	Very High	8	Core
3	US04	Root-cause ranking	Very High	13	Core
4	US06	Remediation recommendation	Very High	8	Core
5	US07	Risk classification	Very High	5	Safety
6	US09	Human approval gate	Very High	5	Safety
7	US10	Audit trail	High	5	Governance
8	US03	Incident timeline	High	5	Usability
9	US05	Deployment correlation	High	8	Analysis
10	US08	Low-risk automation	Very High	8	Automation
11	US12	RBAC	High	5	Security
12	US11	Dashboard	Medium	8	Reporting

7.1 Prioritization Logic

- Safety-related stories are prioritized before autonomous execution.
- Data ingestion and correlation are prerequisites for root-cause analysis.
- Root-cause analysis is prioritized before remediation because actions should be evidence-driven.
- Audit logging is included early so that automation is controlled and traceable from the beginning.
- Dashboard functionality is valuable but can follow the core detection-analysis-remediation flow.

8. Sprint 1 Planning

8.1 Sprint Goal

Build a demonstrable incident-analysis vertical slice that ingests observability data, groups related alerts, creates an incident context, produces a ranked probable root cause, classifies a remediation action by risk, and records the resulting decision in an audit trail.

8.2 Sprint Scope

ID	Story	SP	Sprint Work
US01	Observability ingestion	8	Backend + data pipeline
US02	Alert correlation	8	Correlation service
US03	Incident timeline	5	Incident UI/API
US04	Root-cause ranking	13	Analysis engine
US07	Risk classification	5	Safety rules
US09	Human approval gate	5	Approval workflow
US10	Audit trail	5	Audit service

Total selected story points = 49. This is intentionally a substantial first sprint for a student workshop plan; in a real team, the capacity would be adjusted using historical velocity, team size and availability.

8.3 Sprint Tasks

Story	Actionable Tasks
US01	Configure log/metric connectors; normalize events; validate timestamps; write ingestion tests
US02	Define correlation window; implement grouping rules; test duplicate/related alerts
US03	Create incident model; build timeline API; display ordered events
US04	Prepare feature set; implement scoring logic; expose confidence and evidence
US07	Define low/medium/high rules; implement policy evaluator; test boundary conditions
US09	Create approval request; authorize approver; implement approve/reject states
US10	Create audit schema; record actor/time/action/result; verify audit completeness

9. Story Point Estimation

Fibonacci-style points are used: 1, 2, 3, 5, 8, 13. The points represent relative effort, complexity, uncertainty and integration risk rather than hours.

Story	SP	Estimation Rationale
US01	8	Multiple data formats and connector configuration create integration complexity.
US02	8	Correlation requires time-window logic, grouping rules and duplicate handling.
US03	5	Mostly CRUD/timeline behavior with moderate UI/API work.
US04	13	Highest uncertainty because root-cause ranking combines evidence and analysis logic.
US06	8	Recommendation generation needs action catalog, evidence and risk metadata.
US07	5	Policy rules are clear but safety boundary testing is important.
US09	5	Approval state machine and authorization add moderate complexity.
US10	5	Audit schema and write path are straightforward but completeness is critical.
US11	8	Dashboard combines multiple metrics, charts and filters.
US12	5	RBAC requires roles, permissions and authorization checks.

9.1 Planning Poker Justification

A team can independently select Fibonacci cards, discuss large differences and converge on a shared estimate. For example, US04 may receive 8, 13 and 13 because its uncertainty is higher than ordinary CRUD work; discussion would justify 13.

10. Detailed Incident-Handling Sequence

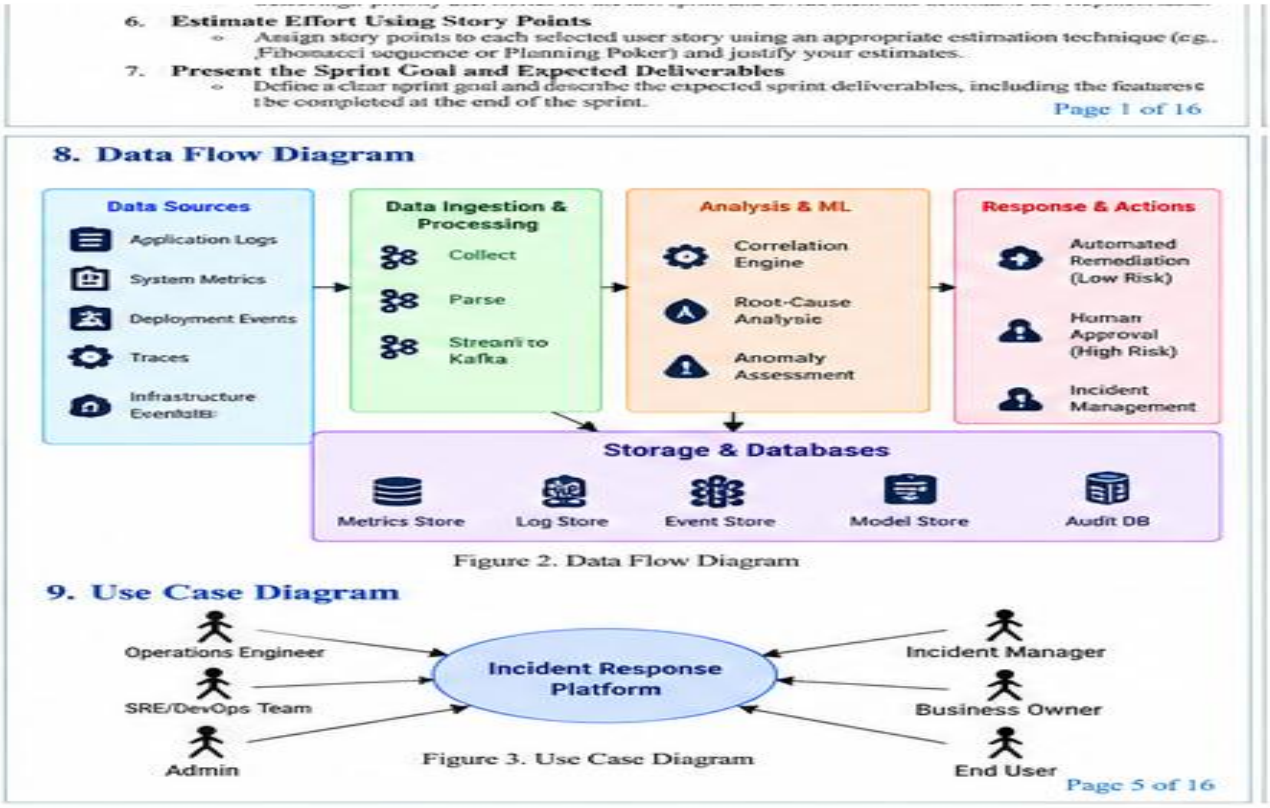


Figure 2. Detailed Incident-Handling Sequence

The sequence illustrates the collaboration among the operator, monitoring system, centralized logs, event-correlation engine, root-cause analysis service, remediation engine, infrastructure layer and incident-management/audit database. High-impact actions pass through a human approval boundary before execution.

10.1 Sequence Interpretation

- Monitoring detects an abnormal condition and sends alert context.
- Logs, metrics and events are ingested and correlated.
- The analysis engine identifies affected services and probable root causes.
- Remediation recommendations are generated with risk information.
- Low-risk actions can proceed within predefined safety limits.
- High-impact actions require an authorized approval response.
- Execution results update the incident and audit records.
- The operator receives resolution information and the incident is closed.

11. Deployment and Infrastructure View

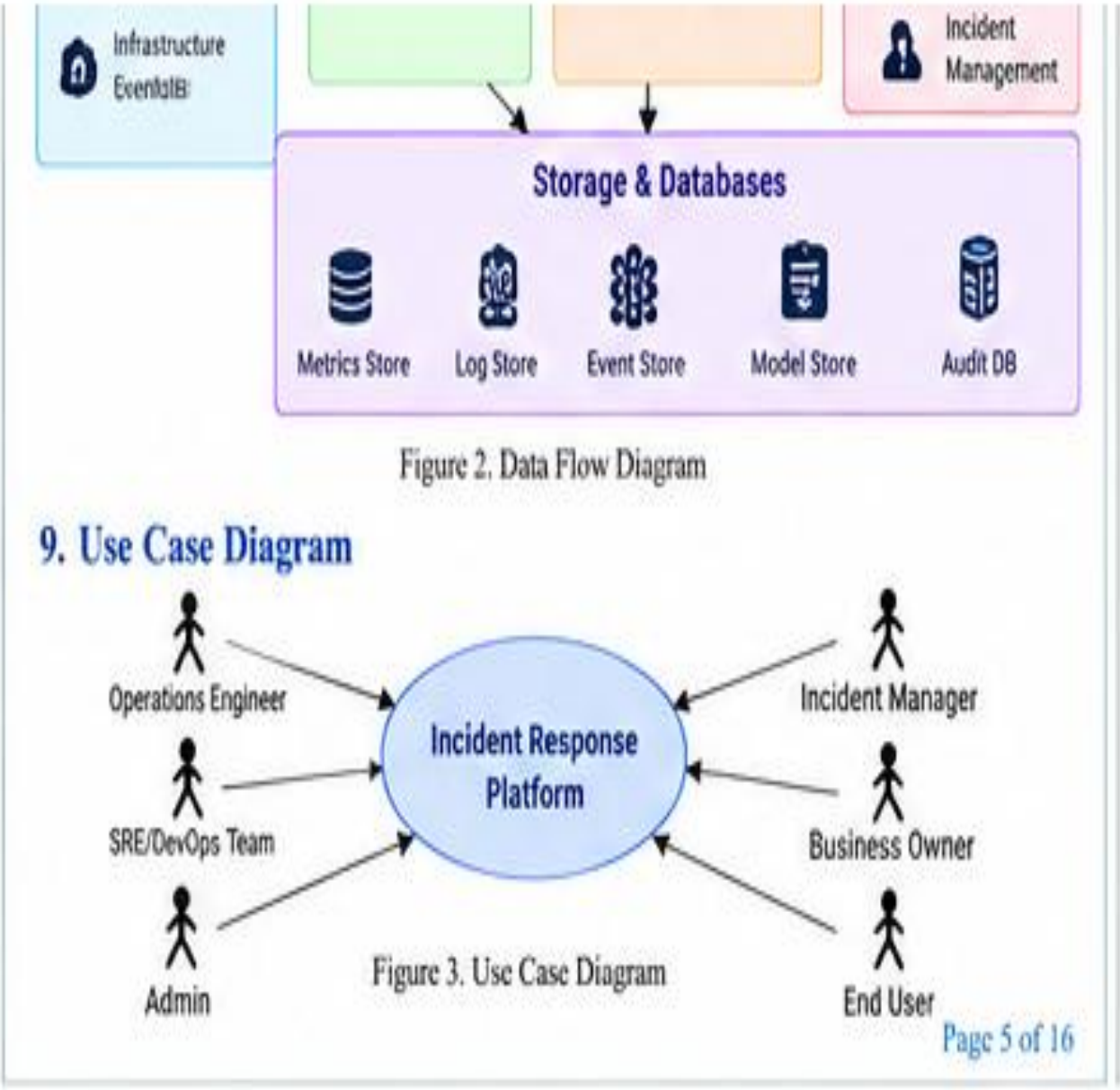


Figure 3. Deployment and Infrastructure View

The deployment view separates client access, front-end applications, API gateway, backend microservices, data stores, cloud infrastructure, supporting services and CI/CD. This separation supports independent scaling and reduces coupling between operational components.

11.1 Infrastructure Components

Layer	Components	Purpose
Client	Web dashboard, mobile app, admin console	Operational access
API	API gateway, authentication	Secure service access
Backend	Incident, ingestion, correlation, RCA, remediation, notification, audit services	Business logic

Data	Operational DB, log store, metrics store, event store, model storage, audit DB	Persistence and analysis
Infrastructure	Virtual network, containers/Kubernetes, nodes, load balancer	Execution and scaling
External	Monitoring, logging, incident management, source control, cloud provider	Integration ecosystem
Delivery	Source control, build/test, deployment pipeline	Continuous delivery

12. Illustrative Incident Dashboard

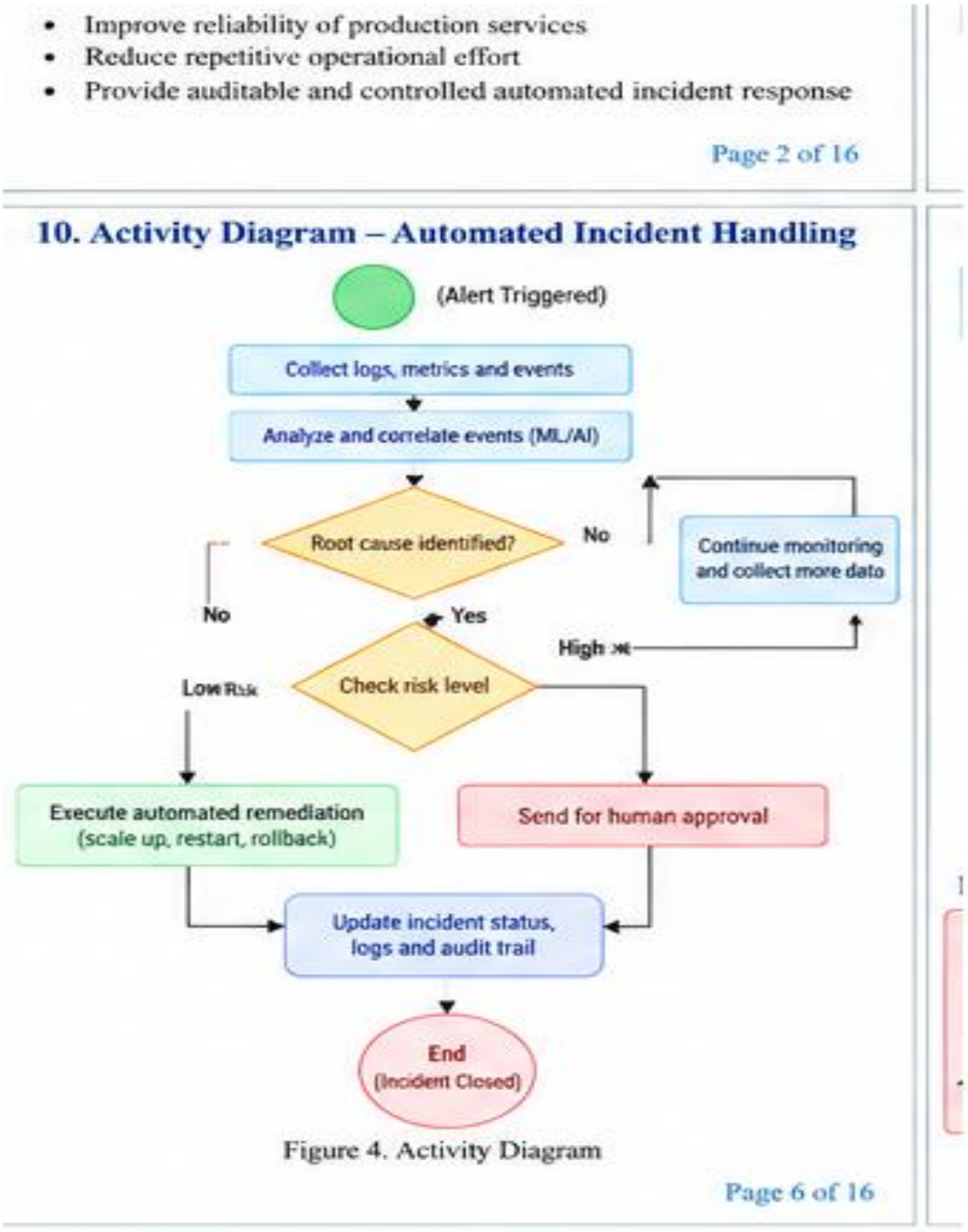


Figure 4. Illustrative Incident Dashboard

The dashboard is designed for operational visibility. It can show total incidents, mean time to detect, mean time to resolve, automated remediation count, service availability, incident trends, status distribution, recent incidents, root-cause progress, service health, remediation actions and live alerts.

12.1 Suggested Dashboard KPIs

KPI	Purpose
Total incidents	Shows incident volume over the selected period.
Mean Time to Detect (MTTD)	Measures how quickly incidents are detected.
Mean Time to Resolve (MTTR)	Measures recovery efficiency.
Auto-remediated incidents	Shows how much safe automation is being used.
Service availability	Shows reliability of production services.
Root-cause confidence	Indicates analysis confidence for active incidents.
Approval pending count	Highlights actions waiting for human decisions.

13. Risk-Based Remediation and Safety Controls

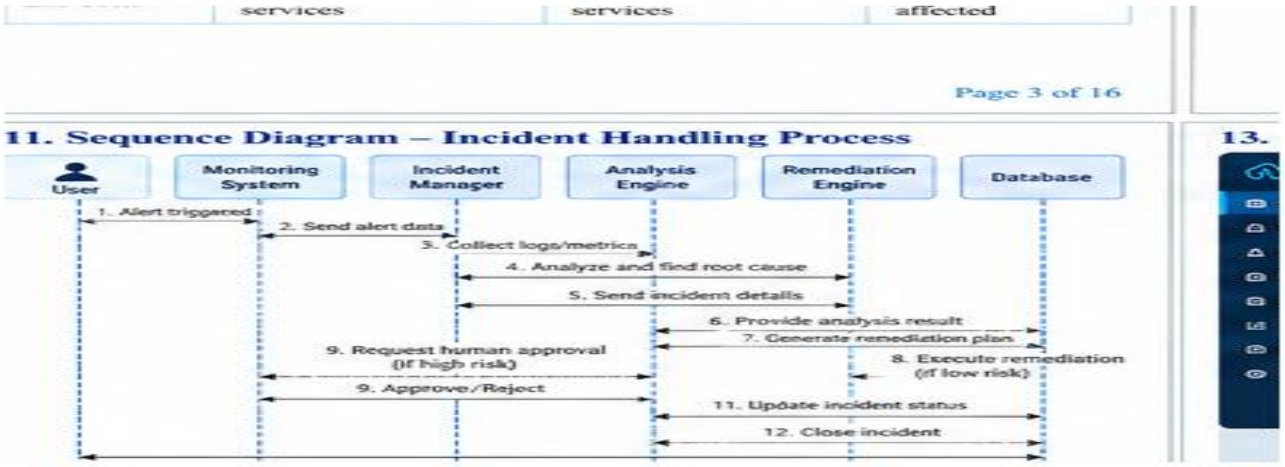


Figure 5. Risk-Based Remediation and Safety Controls

Safety is a central requirement of the problem statement. The platform must not treat every recommended action as safe to execute automatically. Each action is evaluated against impact, likelihood, policy boundaries and authorization requirements.

13.1 Risk Classification

Risk	Example Action	Execution Rule
Low	Restart a stateless application service within approved limits	May auto-execute
Medium	Scale service capacity or restart a dependency	Human approval or policy-specific automation
High	Database rollback, destructive infrastructure change, critical isolation	Manual/authorized approval required
Blocked	Action outside predefined safety boundary	Never auto-execute

13.2 Safety Controls

- Predefined action allow-list.
- Maximum number of automatic retries.
- Resource and scope boundaries.
- Role-based approval permissions.
- Two-person approval for selected critical actions.
- Automatic rollback where technically safe.
- Complete audit logging.
- Emergency stop / disable-automation control.

14. Logical Data Model and Governance

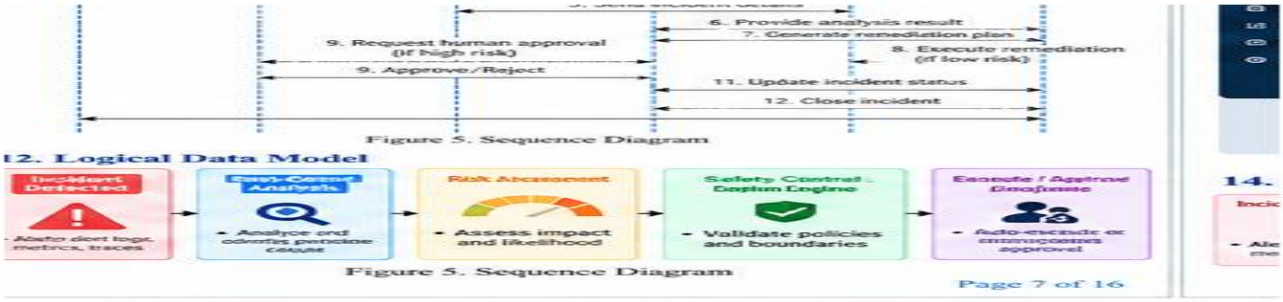


Figure 6. Logical Data Model of the Platform

The logical model links incidents to events, affected services, probable root causes, remediation actions and audit records. This structure supports traceability from the original signal to the final decision and outcome.

Entity	Important Attributes	Purpose
INCIDENT	incident_id, severity, start_time, status	Stores incident lifecycle
EVENT	event_id, incident_id, source, timestamp, type	Stores correlated evidence
SERVICE	service_id, name, criticality	Represents affected services
ROOT_CAUSE	cause_id, incident_id, description, confidence	Stores probable causes
ACTION	action_id, incident_id, risk_level, status	Stores remediation decisions
AUDIT	audit_id, actor, timestamp, action, result	Stores governance evidence

15. Incident Lifecycle

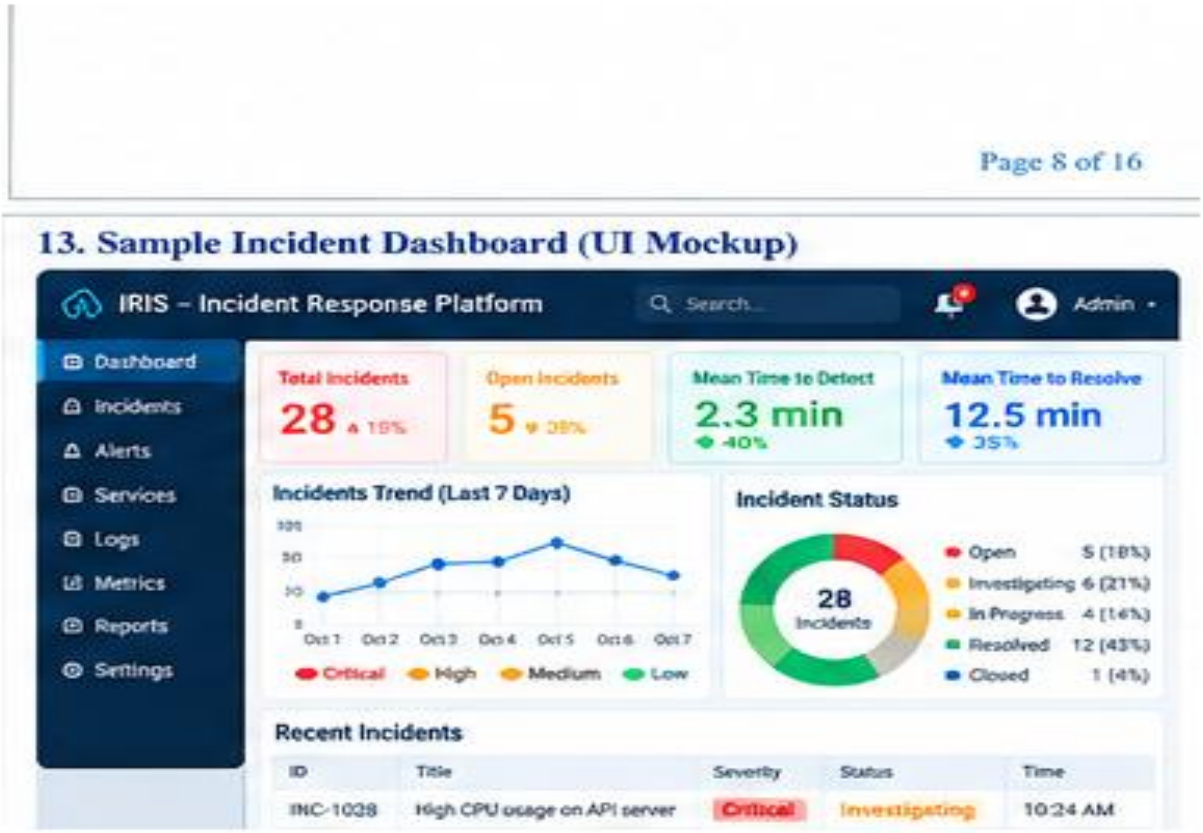


Figure 7. Sample Incident Lifecycle

The incident lifecycle begins with detection, continues through analysis and remediation, then moves to verification and closure. A complete lifecycle prevents incidents from being considered resolved merely because an action was executed; service health should be verified before closure.

Stage	Key Activity	Exit Condition
Detection	Create incident from alert correlation	Incident acknowledged
Analysis	Collect evidence and identify probable cause	Cause ranked
Remediation	Recommend and execute/approve action	Action completed
Verification	Check service health and recurrence	Recovery confirmed
Closure	Document outcome and close incident	Audit complete

16. Sprint Backlog and Task Board

The first sprint can be represented on a simple Kanban-style board so that progress is visible.

To Do	In Progress	Review / Test	Done
US04 – RCA scoring	US01 – ingestion	US07 – risk rules	Sprint setup
US02 – correlation	US03 – timeline	US10 – audit	Repository setup
US09 – approval	—	—	Architecture baseline

16.1 Definition of Done

- Code is implemented and peer reviewed.
- Unit and integration tests pass.
- Acceptance criteria are demonstrated.
- Security and authorization checks are included.
- Audit records are generated where applicable.
- Documentation is updated.
- Feature is deployable in the sprint environment.
- Product Owner accepts the completed story.

17. Sprint Deliverables

Deliverable	Description	Expected Evidence
Ingestion prototype	Receives normalized metrics/log events	Sample event records
Correlation module	Groups related alerts	Correlation test results
Incident context	Shows timeline and affected services	Incident screen/API response
RCA prototype	Ranks probable causes	Confidence and evidence output
Risk engine	Classifies remediation actions	Policy test cases
Approval workflow	Blocks high-impact actions until approval	Approval/rejection records
Audit trail	Records actions and decisions	Audit database entries
Sprint demonstration	Shows complete vertical flow	Demo script/video/screens

17.1 Sprint Review

The Sprint Review should demonstrate one representative incident from alert detection through correlation, probable-cause analysis, remediation recommendation, safety classification, approval where required, action result and audit record.

17.2 Sprint Retrospective

The team should review what worked, what caused delays, which estimates were inaccurate, whether stories were too large, and which engineering practices should change in the next sprint.

18. Agile Release Roadmap

Sprint	Primary Goal	Major Deliverables
Sprint 1	Core incident-analysis vertical slice	Ingestion, correlation, incident context, RCA prototype, safety rules, audit
Sprint 2	Operational integrations	Deployment correlation, incident-management integration, notifications
Sprint 3	Controlled automation	Low-risk automation, approval workflows, rollback support
Sprint 4	Operational dashboard	Dashboard, KPI reporting, trend analysis
Sprint 5	Hardening	Security, load testing, reliability testing, observability
Sprint 6	Production readiness	Deployment automation, documentation, training and final acceptance

19. Risks, Mitigation and Governance

Risk	Impact	Likelihood	Mitigation
Incorrect root-cause recommendation	High	Medium	Show evidence/confidence; keep human review for uncertain cases
Unsafe automated remediation	Very High	Low/Medium	Allow-list, safety boundaries, approvals, rollback
Alert correlation errors	High	Medium	Tune rules and evaluate historical incidents
Integration failure	Medium	Medium	Adapters, retries, health checks
Audit gaps	High	Low	Central audit service and automated validation
Unauthorized action	Very High	Low	RBAC, least privilege, approval controls
Model drift / false confidence	High	Medium	Monitoring, threshold tuning, fallback rules

19.1 Governance Principle

Automation should be explainable, bounded and reversible wherever possible. The system should expose why an action was recommended, which policy allowed it, who approved it when approval was required, what was executed, and what the resulting service state was.

20. Requirements Traceability

Problem Requirement	Epic	User Story	Acceptance Evidence
Correlate monitoring data	E1/E2	US01/US02	Normalized events + grouped incidents
Correlate deployment changes	E3	US05	Deployment linked to incident timeline
Use incident histories	E3/E6	US04/US03	Historical incident evidence
Identify probable causes	E3	US04	Ranked causes + confidence
Recommend safe remediation	E4/E5	US06/US07	Action + risk classification
Automate low-risk recovery	E5	US08	Successful controlled execution
Human approval for high-impact	E5	US09	Approval gate
Maintain audit trails	E7	US10	Complete audit record

21. Expected Outcomes and Measurement

The problem statement specifies reductions in alert overload, improved root-cause identification, reduced incident response and recovery time, improved reliability, reduced repetitive effort and auditable automation. These outcomes can be measured using the following indicators.

Outcome	Metric	Target Direction
Reduce alert overload	Alerts per incident / duplicate alert ratio	Decrease
Improve root-cause identification	Correct top-ranked cause rate	Increase
Reduce response time	MTTD	Decrease
Reduce recovery time	MTTR	Decrease
Improve reliability	Availability / error rate	Availability up, errors down
Reduce repetitive effort	Manual remediation actions per incident	Decrease
Improve auditability	Percentage of actions with complete audit record	Approach 100%

21.1 Definition of Success

The project is successful when the prototype can demonstrate an incident from detection to closure, identify a plausible cause from correlated evidence, recommend a remediation action, prevent unsafe automatic execution, route high-impact actions for approval, and preserve an auditable record of the complete decision path.

22. Final Sprint Goal and Expected Deliverables

Sprint Goal: Deliver a secure end-to-end prototype that converts raw monitoring signals into a correlated incident, produces a probable root cause, recommends a risk-classified remediation action, applies the appropriate automation or approval rule, and records the complete incident and decision history.

Category	Sprint Deliverable
Core platform	Event ingestion and normalization
Incident management	Alert correlation and incident context
Analysis	Probable root-cause ranking
Safety	Risk classification and policy evaluation
Human control	Approval workflow for high-impact actions
Governance	Audit trail
Demonstration	End-to-end incident scenario

The selected sprint scope is aligned with the assessment requirement to select high-priority user stories for the first sprint, divide them into actionable development tasks, estimate their effort, and present a clear sprint goal and deliverables. □filecite□turn4file0□L46-L56□

Rubrics for Calculation

Criteria	Excellent (5 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Project Vision & Stakeholder Identification (30%)	Project vision is clear, comprehensive, and aligned with the problem statement. All relevant stakeholders and their roles are correctly identified.	Vision is clear with minor omissions. Most stakeholders are correctly identified.	Vision is partially defined. Some stakeholders are missing or incorrectly identified.	Vision is unclear or incomplete. Stakeholders are poorly identified or missing.
Epics & User Stories (25%)	Epics are well-defined and user stories are complete, follow Agile format, and fully capture functional requirements.	Epics and user stories are mostly complete with minor formatting or requirement issues.	User stories are partially complete with limited Agile structure.	User stories are incomplete, unclear, or do not follow Agile practices.
Acceptance Criteria (20%)	Acceptance criteria are specific, measurable, testable, and aligned with all user stories.	Acceptance criteria are mostly clear and testable with minor gaps.	Acceptance criteria are partially defined and lack clarity or completeness.	Acceptance criteria are missing or not aligned with user stories.
Product Backlog Prioritization & Sprint Planning (15%)	Product backlog is logically prioritized, sprint backlog is realistic, and task breakdown is complete.	Backlog prioritization and sprint planning are mostly appropriate with minor issues.	Prioritization or sprint planning is partially complete with limited justification.	Backlog prioritization and sprint planning are poorly organized or incomplete.
Story Point Estimation, Sprint Goal & Presentation (10%)	Story point estimation is well justified, sprint goal is clear, deliverables are realistic, and presentation is professional and well organized.	Estimation and sprint goal are mostly appropriate with minor inconsistencies. Presentation is clear.	Estimation or sprint goal lacks justification. Presentation is adequate but incomplete.	Estimation is unrealistic or missing. Sprint goal, deliverables, and presentation are unclear or incomplete.

Criteria	Mark
Project Vision & Stakeholder Identification (30%)	/5
Epics & User Stories (25%)	/5
Acceptance Criteria (20%)	/5
Product Backlog Prioritization & Sprint Planning (15%)	/5
Story Point Estimation, Sprint Goal & Presentation (10%)	/5
TOTAL	/25

23. Detailed Product Backlog Breakdown

The backlog is decomposed further so that each epic can be implemented, tested and reviewed independently. The decomposition also makes dependencies visible before sprint commitment.

Epic	Feature	User Stories	Dependency	Definition of Ready
E1	Observability ingestion	US01	Source credentials/config	Source identified + sample payload
E2	Correlation	US02, US03	E1	Correlation rule + test data
E3	Root-cause analysis	US04, US05	E1 + E2	Evidence fields defined
E4	Remediation recommendation	US06	E3	Action catalog defined
E5	Controlled automation	US07, US08, US09	E4	Safety policy approved
E7	Audit governance	US10	E5	Audit fields finalized
E8	Dashboard	US11	E2/E3/E5	KPI definitions agreed
E9	Security	US12	Platform architecture	Roles and permissions defined

24. User Story Mapping

User Story Mapping organizes the platform from the user's journey rather than from technical components. The backbone is Detect → Understand → Decide → Act → Verify → Learn.

Journey	Detect	Understand	Decide	Act	Verify	Learn
Primary user activity	Receive alert	Correlate evidence	Evaluate cause/risk	Approve or execute	Check service health	Review history
MVP story	US01	US02 + US03	US04 + US07	US06 + US09	US10	US10
Later enhancement	Predictive detection	Dependency graph	Confidence calibration	Advanced automation	Automatic rollback	Trend analytics

The map prevents the team from spending the first sprint entirely on infrastructure while leaving the operational user journey incomplete. The MVP should demonstrate one complete vertical slice across the backbone.

25. Acceptance Criteria Quality Check

Acceptance criteria should be specific enough that a developer can implement the story and a tester can determine pass/fail without interpretation.

Quality Check	Example for US09 – Human Approval	Pass Condition
Specific	High-impact action creates approval request	Exact event is defined
Measurable	Execution count remains zero before approval	System logs show zero execution
Testable	Approve and Reject produce different states	Automated test passes
Secure	Only authorized approver can approve	Unauthorized request is rejected
Auditable	Decision is recorded with actor/time	Audit record is complete

26. Definition of Ready for Sprint Planning

- The user story has a clear business value.
- The story has acceptance criteria.
- Dependencies are identified.
- Required data or test inputs are available.
- Security and safety implications are understood.
- The story is small enough to complete within the sprint.
- The Product Owner and development team understand the expected outcome.
- The story can be demonstrated at the end of the sprint.

27. Sprint Capacity and Commitment

Story points are relative measures. A team should not treat one point as a fixed number of hours. Sprint commitment should be based on historical velocity, available team members, planned leave, technical uncertainty and operational support work.

Factor	Example Planning Effect
Team availability	Reduce commitment when members are unavailable
Historical velocity	Use average completed points from previous sprints
Technical uncertainty	Reserve capacity for spikes and investigation
Integration work	Reserve capacity for external system testing
Security testing	Include time for authorization and safety validation
Bug fixing	Keep a small capacity buffer

For a student project without historical velocity, the 49-point Sprint 1 selection should be treated as a planning exercise rather than a guaranteed delivery commitment. A real team would calibrate the commitment after one or two sprints.

28. Sprint Planning Meeting Agenda

Time	Activity	Output
0–10 min	Review product vision and sprint goal	Shared objective
10–20 min	Review top backlog stories	Candidate stories
20–35 min	Clarify acceptance criteria	Ready stories
35–50 min	Estimate effort	Story points
50–65 min	Break stories into tasks	Task list
65–75 min	Check dependencies and capacity	Feasible sprint
75–90 min	Confirm commitment	Sprint backlog

29. Agile Roles and Responsibilities

Role	Responsibilities in This Project
Product Owner	Prioritize backlog, clarify business value, accept completed stories
Scrum Master	Facilitate sprint events, remove impediments, protect Agile process
Backend Developer	Build ingestion, correlation, incident and remediation services
ML/Analysis Engineer	Develop root-cause scoring and evidence ranking
Frontend Developer	Build dashboard, incident views and approval screens
DevOps/SRE	Configure deployment, monitoring, CI/CD and operational tests
Security Engineer	Define RBAC, approval controls, audit and safety policies
Tester/QA	Validate acceptance criteria, integration, safety and regression behavior

30. Testing Strategy

Testing must cover both normal incident handling and failure/safety scenarios. Because the system can recommend or execute production actions, safety testing is as important as functional testing.

Test Level	Examples	Expected Result
Unit Testing	Correlation rules, risk classifier, RCA scoring	Correct output for known inputs
Integration Testing	Monitoring → ingestion → incident creation	Events propagate correctly
Workflow Testing	Incident → recommendation → approval	Correct state transitions
Security Testing	Unauthorized approval, privilege escalation	Requests blocked
Safety Testing	High-impact action under automation	Execution prevented
Recovery Testing	Action fails or rollback occurs	Incident remains traceable
Performance Testing	High alert volume	System remains responsive
User Acceptance Testing	SRE executes sample incident	Workflow understandable and usable

31. Non-Functional Requirements

Requirement	Target / Expectation
Availability	Platform should remain available during incidents; critical components should avoid single points of failure.
Performance	Incident ingestion and correlation should provide results quickly enough to support operational response.
Security	Authentication, RBAC, least privilege and secure service communication.
Auditability	Every approval and automated action must have traceable records.
Scalability	Ingestion and analysis services should scale independently with event volume.
Maintainability	Services should be modular, documented and testable.
Usability	Incident status, evidence, risk and next action should be visible without searching multiple systems.
Recoverability	Failed automation should not leave the incident state ambiguous.

32. API and Integration Requirements

The platform is intended to integrate with existing enterprise tools. Integration contracts should be defined before implementation so that changes in one system do not propagate uncontrolled changes throughout the platform.

Integration	Data Exchanged	Direction	Purpose
Monitoring	Metrics, alerts, service health	Inbound	Detect abnormal behavior
Logging	Structured/unstructured logs	Inbound	Evidence collection
Deployment	Release/change events	Inbound	Change correlation
Incident Management	Incident status, assignment	Bi-directional	Operational workflow
Infrastructure-as-Code	Approved infrastructure actions	Bi-directional	Controlled remediation
Identity Provider	Users, roles, authentication	Inbound	Access control
Notification	Approval and resolution messages	Outbound	Human communication

33. Security Architecture

Security controls should be designed into the platform rather than added after automation is implemented. The most sensitive capability is the remediation engine because it can influence production state.

Control	Implementation Approach
Authentication	Central identity provider / SSO
Authorization	Role-based access control with least privilege
Approval	Explicit approval permission for high-impact actions
Secrets	Centralized secret management; no credentials in source code
Network Security	Private service communication and restricted ingress
Audit	Append-only action and approval records
Action Policy	Allow-list and safety boundary checks before execution
Emergency Stop	Global switch to disable automated remediation
Data Protection	Encryption in transit and at rest where applicable

34. Safety Policy Matrix

Action Category	Example	Automation	Approval	Rollback
Low	Restart stateless service	Allowed within policy	No	Preferred
Medium	Scale service	Conditional	Required by policy	Preferred
High	Database rollback	Blocked by default	Mandatory	Required
Destructive	Delete infrastructure/resource	Blocked	Manual change process	Mandatory
Unknown	Action not in allow-list	Blocked	Manual investigation	Not applicable

35. Root-Cause Analysis Approach

The platform should not depend on one signal. Root-cause analysis combines temporal correlation, service dependency information, deployment changes, historical incident similarity and health indicators.

- Temporal evidence: determine which signals changed before and after incident onset.
- Dependency evidence: identify whether an upstream or downstream service explains the symptom.
- Change evidence: compare recent deployments, configuration changes and infrastructure modifications.
- Historical evidence: compare the incident with previously resolved incidents.
- Confidence scoring: rank candidate causes and show supporting evidence.
- Fallback reasoning: if confidence is low, recommend investigation rather than unsafe automation.

36. Remediation Decision Logic

A remediation recommendation should be treated as a decision pipeline rather than a single AI prediction.

Decision Stage	Question	Outcome
Evidence sufficiency	Do we have enough reliable evidence?	Continue / collect more data
Cause confidence	Is a probable cause sufficiently supported?	Rank cause / investigate
Action validity	Is the proposed action applicable?	Candidate action / reject
Risk level	What production impact could occur?	Low / Medium / High
Policy check	Is this action allowed?	Allow / approval / block
Execution	Can the action be safely executed?	Execute / wait
Verification	Did service health recover?	Close / rollback / continue investigation

37. Incident Metrics and KPIs

Metric	Formula / Meaning	Use
MTTD	Detection time − incident start	Measure detection efficiency
MTTR	Resolution time − incident start	Measure recovery efficiency
Alert reduction	1 − correlated alerts / raw alerts	Measure noise reduction
Top-cause accuracy	Correct top-ranked causes / evaluated incidents	Measure RCA quality
Automation success	Successful automated actions / automated actions	Measure automation reliability
Approval latency	Approval time − approval request time	Measure human bottleneck
Rollback rate	Rollback actions / automated actions	Detect unsafe recommendations
Audit completeness	Actions with complete audit / total actions	Measure governance

38. Release Acceptance Checklist

- All critical user stories satisfy acceptance criteria.
- No high-impact remediation can execute without required approval.
- Every remediation attempt creates an audit record.
- Unauthorized users cannot approve or execute protected actions.
- Incident timelines contain required evidence.
- Root-cause results expose confidence/evidence information.
- Integration failures are visible and recoverable.
- Dashboard KPIs match underlying records.
- Regression tests pass.
- Product Owner approves the release candidate.

39. Final Assessment Mapping

Assessment Requirement	Where Addressed
Project Vision and Stakeholders	Sections 1–2, 5–6
Epics and User Stories	Sections 4–5, 23–24
Acceptance Criteria	Section 6 and 25
Product Backlog Prioritization	Section 7 and 23
Sprint Backlog	Section 8
Actionable Tasks	Section 8.3
Story Point Estimation	Section 9 and 27
Sprint Goal	Section 8.1 and 22
Expected Deliverables	Sections 17 and 22
Professional Presentation	Tables, diagrams, workflow figures and dashboard

40. Comprehensive Conclusion

The Autonomous Incident Response and Root-Cause Analysis Platform is structured as an Agile, safety-controlled operational system rather than as an unrestricted automation tool. The project begins with a clear business problem: large volumes of observability signals make incident diagnosis slow and fragmented. The proposed backlog converts this problem into manageable capabilities for ingestion, correlation, root-cause analysis, remediation recommendation, controlled automation, governance and reporting.

The first sprint is intentionally designed as a vertical slice. Instead of delivering isolated technical components, the team demonstrates the complete path from an incoming signal to a correlated incident,

probable root cause, risk assessment, approval or controlled action, and audit record. This approach provides early stakeholder feedback and exposes integration risks before later sprints.

The strongest engineering principle is controlled autonomy. Low-risk actions may be automated only inside predefined boundaries, while high-impact actions require human authorization. This balances the operational benefit of automation with production safety, accountability and auditability.

The Agile plan also creates measurable outcomes. Alert volume, MTTD, MTTR, root-cause accuracy, automation success, approval latency, rollback rate and audit completeness can be monitored to evaluate whether the system delivers the intended business value.

Overall, the proposed sprint plan is aligned with the assessment learning outcome because it applies Agile project-management techniques to a realistic software engineering problem and translates a complex system requirement into a prioritized, estimable and demonstrable product backlog.

Rubrics for Calculation

Criteria	Excellent (5 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Project Vision & Stakeholder Identification (30%)	Project vision is clear, comprehensive, and aligned with the problem statement. All relevant stakeholders and their roles are correctly identified.	Vision is clear with minor omissions. Most stakeholders are correctly identified.	Vision is partially defined. Some stakeholders are missing or incorrectly identified.	Vision is unclear or incomplete. Stakeholders are poorly identified or missing.
Epics & User Stories (25%)	Epics are well-defined and user stories are complete, follow Agile format, and fully capture functional requirements.	Epics and user stories are mostly complete with minor formatting or requirement issues.	User stories are partially complete with limited Agile structure.	User stories are incomplete, unclear, or do not follow Agile practices.
Acceptance Criteria (20%)	Acceptance criteria are specific, measurable, testable, and aligned with all user stories.	Acceptance criteria are mostly clear and testable with minor gaps.	Acceptance criteria are partially defined and lack clarity or completeness.	Acceptance criteria are missing or not aligned with user stories.
Product Backlog Prioritization & Sprint Planning (15%)	Product backlog is logically prioritized, sprint backlog is realistic, and task breakdown is complete.	Backlog prioritization and sprint planning are mostly appropriate with minor issues.	Prioritization or sprint planning is partially complete with limited justification.	Backlog prioritization and sprint planning are poorly organized or incomplete.
Story Point Estimation, Sprint Goal & Presentation (10%)	Story point estimation is well justified, sprint goal is clear, deliverables are realistic, and presentation is professional and well organized.	Estimation and sprint goal are mostly appropriate with minor inconsistencies. Presentation is clear.	Estimation or sprint goal lacks justification. Presentation is adequate but incomplete.	Estimation is unrealistic or missing. Sprint goal, deliverables, and presentation are unclear or incomplete.

Criteria	Mark
Project Vision & Stakeholder Identification (30%)	/5
Epics & User Stories (25%)	/5
Acceptance Criteria (20%)	/5
Product Backlog Prioritization & Sprint Planning (15%)	/5
Story Point Estimation, Sprint Goal & Presentation (10%)	/5
TOTAL	/25